# CIS11 Course Project

**Beep-Boop**
**D'von White, Ernie Gamez, Leonardo Garcia**
**Test Score Calculator**
**05/24/2026**

**Advisor: Kasey Nguyen, PhD**

Part 1
Sections:

## Program Objectives:

The core purpose of this project is to plan, design, and build a successful LC-3 assembly language program that applies fundamental course concepts in computer architecture, data processing, and hardware-level programming. Specifically, this application acts as a hardware-constrained Test Score Calculator that automates low-level statistical mapping. The system accepts a sequential matrix of 5 student test scores via terminal keyboard input, processes the metrics natively within its registers, and displays the isolated minimum score, maximum score, cumulative integer average, and corresponding letter grade output directly onto the console interface.

## Program Functionality and Structure

The application replaces abstract, high-level computational layers with a low-overhead software routine directly interacting with the LC-3 simulation hardware platform. The workflow maps across a precise linear processing lifecycle:

1. System Entry: The program boots up, initializes its hardware stack baseline, and allocates an empty 5-word memory array slot using a base pointer.
2. Synchronous Ingestion: The user is prompted sequentially via console display strings to type numeric test values.
3. Data Encoding Translation: The key clicks are captured as ASCII byte values, instantly transformed into pure binary data equivalents via programmatic subtraction, and loaded sequentially into the memory array slots.
4. Statistical Calculation: Modular computation blocks evaluate array elements using bitwise comparisons and subtraction loops to isolate extremes and determine sum totals.
5. Output Report Delivery: Binary integers are parsed, re-converted back into textual ASCII sequences, and rendered neatly onto the user terminal display screen.

## Testing ()

"The application’s calculations and logic are verified using a fixed test suite containing the exact data points: 52, 87, 96, 79, and 61"
(A version of your pseudocode has been turned into a program for this section)

**Part 2**

Sections:

Architectural Scope and Processing Phases

Terminology & System Environment

The application transitions from user-space logic to hardware-level execution by interfacing directly with the LC-3 operating system via standard service call directives, implemented as TRAP routines. Rather than writing low-level device drivers to manually poll hardware registries for keyboard status or console video matrices, the software utilizes these standardized vectors to safely delegate Input/Output (I/O) control to the operating system layer.

Specifically, the program invokes:

1. GETC (TRAP x20): to halt execution and scan for a single character input from the keyboard buffer.
2. OUT (TRAP x21): to push processed ASCII character sequences out to the user display terminal.
3. PUTS (TRAP x22): to loop through memory arrays and write full strings of system prompts and calculated outputs sequentially.
4. HALT (TRAP x25): to safely park the machine's program counter upon execution completion, preventing the processor from running into unallocated memory addresses.

## Program Scope and Sequential Processing Phases (l)

The implementation lifecycle follows a structured five-phase development blueprint to ensure low-level hardware alignment. During Phase 1, the architecture handles fundamental program setup by declaring the logical code origin, initializing memory allocations, building data arrays, defining constant labels, and setting up the runtime stack pointers and basic system call configurations. Phase 2 manages data ingestion, capturing five individual student test scores from the console keyboard, running them through conversion routines, and writing them sequentially into memory using direct pointer addressing. Phase 3 implements core computational processing by using iterative loops and conditional branching to determine the sample minimum, maximum, total sum, and cumulative average. Phase 4 handles the logical conversion of the final average into an academic grade character code using sequential register data comparisons. Finally, Phase 5 integrates modular subroutine architectures and stack mechanics, finalizing the low-level execution paths and protecting execution states.

## Architectural Performance Metrics and Resource Safeguards (l)

To maintain optimal execution efficiency inside the LC-3 simulation shell, the application relies on strategic hardware-level processing constraints rather than relying on heavy memory virtualization. The system restricts its active data footprint by allocating an array of exactly five test scores, keeping memory overhead to an absolute minimum and enabling math modules to complete tracking calculations instantaneously upon final character ingestion. By utilizing direct

pointer addressing to step through consecutive memory blocks, the program entirely removes the need for redundant memory-to-register caching cycles. Furthermore, modular data organization through subroutines and dedicated stack tracking guarantees clean register management throughout execution, keeping critical variables tightly packed and easily accessible.

## User Interface Mechanics and System Usability (l)

System communication and readability are driven by predictable, interactive terminal design constraints. The program outputs descriptive string prompts before every entry cycle to explicitly notify the user which test index is currently being requested and to clarify the exact required format of incoming numbers. The software is intentionally built around a clean, single-purpose calculation stream, processing numbers immediately upon full matrix completion to prevent terminal input hang-ups. Additionally, the underlying source assembly code enforces clear formatting discipline, categorizing structural functions into clearly commented blocks and labeled routines so outside developer audits can effortlessly trace execution paths from start to finish.

## System Memory Architecture & Storage Allocation

The application is structured around a highly disciplined memory map layout designed to guarantee predictable execution behavior within the LC-3 simulation shell. The program structure relies on specialized assembler directives to partition specific address spaces for code, constants, data containers, and communication boundaries. The operational core of the machine instructions initializes directly at address x3000 via the program origination directive (.ORIG x3000). This vector places the entry-point executable instructions safely within user space, completely clear of the LC-3 system's architecture traps, trap vector tables (x0000 to x01FF), and operating system supervisor data blocks.

Immediately following the executable loops, fixed hexadecimal and decimal parameters are burned into explicitly labeled memory slots using .FILL directives. These slots are allocated to store tracking components like the negative ASCII mapping masks (xFFD0), loop limit counters (#-5), grade range boundary scores, and raw character literals (x0041 for 'A') so they can be loaded cleanly using PC-relative offset instructions. Instead of scattering inputs across unorganized registers, a contiguous 5-word block of storage is reserved via a Block Storage of Words (.BLKW 5) command. Labeled as SCORE_ARRAY, this structure preserves five individual 16-bit cells in sequential memory to establish a robust raw data matrix. Finally, text communication paths are established through fixed, null-terminated string vectors (.STRINGZ). This design feeds base address labels directly into PUTS routines, storing the visual terminal menus and data descriptions cleanly alongside user input storage frames to maintain strict structural data isolation.

## Terminal Core Outputs & Report Metrics

The primary processing objective of the system is the systematic computation and immediate delivery of raw statistical metrics rendered cleanly onto the console terminal interface. Once data ingestion, array sorting, and grade evaluation routines reach structural termination, the program transitions into an output reporting cycle by clearing working registers and retrieving the calculated integer properties from their designated memory cells:

A. **Program Origination (.ORIG x3000)**: The operational core of the machine instructions initializes directly at address x3000. This vector places the entry-point executable instructions safely within user space, completely clear of the LC-3 system's architecture traps, trap vector tables (x0000 to x01FF), and operating system supervisor data blocks.

B. **Static Data Constants (.FILL)**: Fixed hexadecimal and decimal parameters are burned into explicitly labeled memory slots immediately after the execution loop. These slots are allocated to store tracking components like the negative ASCII mapping masks (xFFD0), loop limit counters (#-5), grade range boundary scores, and raw character literals (x0041 for 'A') so they can be loaded cleanly using relative offset instructions.

C. **Static Score Array (.BLKW 5)**: Instead of scattering inputs across unorganized registers, a contiguous 5-word block of storage is reserved via a Block Storage of Words command. Labeled as SCORE_ARRAY, this structure preserves five individual 16-bit cells in sequential memory to establish a robust raw data matrix.

D. **Console I/O Communication Buffers (.STRINGZ)**: Text communication paths are established through fixed, null-terminated string vectors. This design feeds base address labels directly into PUTS routines, storing the visual terminal menus and data descriptions cleanly alongside user input storage frames to maintain strict structural data isolation.

## Low-Level Instruction Set Architecture Optimization

To achieve maximum computational efficiency with minimal clock-cycle overhead, the program utilizes a tightly optimized instruction mix from the LC-3 Reduced Instruction Set Computer architecture. The codebase deliberately balances three fundamental opcode categories to achieve high-density execution control without wasting hardware processing power. Arithmetic operations such as ADD and NOT handle running tally accumulations and pointer offset logic, while bitwise inversion routines combined with an incremental add form the backbone of two's complement mathematical negation for subtraction comparisons. Data movement operations leverage commands like LD, ST, LDR, STR, and LEA to target global variable values using relative load/store instructions, process structural base arrays via relative indirect registers, and

resolve runtime console pointer variables using load-effective address calculations. Finally, conditional operations using BRn, BRz, and BRp manage program control entirely by monitoring the state of the processor’s status flag registers. This instruction routing allows the application to construct loop boundaries for array traversals and implement complex branching for grade-scale evaluation structures.

## Hardware Overflow Protections & Allocation Constraints

Operating on a 16-bit word-addressable microarchitecture requires strict limits on data boundaries to prevent register wrap-around errors and unshielded memory corruption. Storage allocation is tightly bounded by reserving an absolute static space of five sequential 16-bit slots, isolating the program data entirely from adjacent code boundaries and preventing memory clobbering.  To protect calculation integrity, the math modules track potential arithmetic overflows during the summation loop. Because each test entry is capped at a maximum value of 100, the accumulation register scales to an absolute maximum weight of 500 (5 \times 100), staying well within the $+32,767$ upper boundary for signed 16-bit integers. This structural safeguard keeps the most significant bit safely at zero, preserving correct mathematical behavior during high-speed iterative evaluations.

## Algorithmic Data Ingestion & ASCII Conversion Processing

Because the LC-3 hardware architecture handles terminal keyboard inputs and console prints purely through binary text representations, the application features an integrated ASCII translation layer. When a user types a character digit on the keyboard, the machine intercepts a raw ASCII value where characters '0' through '9' map to hexadecimal bytes x0030 through x0039. The translation layer manages data processing across two specific directions. In the input normalization loop, the system strips the text alignment code by adding a two's complement mask of -x0030 (xFFD0), transforming the incoming character byte into a raw calculating integer value stored directly in the score matrix. Conversely, the output reconstruction engine handles text rendering by passing binary calculation values through an iterative base-10 division loop. The resulting tens and units digits are re-offset by adding +x0030 to match standard text codes before being passed to console output routines.

## Part 3: Appendices

Sections:
A. Drafting and Inception

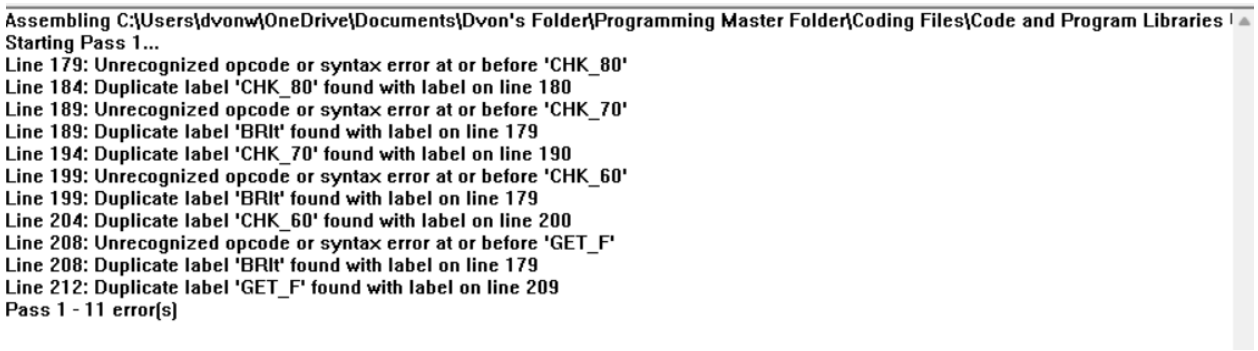
A-1: Test Calculator Pseudocode

```
START PROGRAM

// initialize stack pointer and variables
Initialize STACK
Set SUM = 0
Set COUNT = 5
Set INDEX = 0

// create array for 5 test scores
Create SCORE_ARRAY[5]

// call input subroutine
Call INPUT_SUBROUTINE

// input subroutine
INPUT_SUBROUTINE

FOR INDEX = 0 to 4

        // prompt user for score
        Display “Enter Test Score: “

        // read keyboard input
        Read input characters

        // convert ASCII characters into integer value
        Convert ASCII input to number

        // store score into array using pointer
        Store score into SCORE_ARRAY[INDEX]
END FOR


RETURN to MAIN PROGRAM


// initialize min and max using first score
Set MIN = SCORE_ARRAY[0]
Set MAX = SCORE_ARRAY[0]
```

```
// reset index for calculations
Set INDEX = 0

// call calculation subroutine
Call CALCULATION_SUBROUTINE

// calculation subroutine
CALCULATION_SUBROUTINE


FOR INDEX = 0 to 4

        // load current score
        CURRENT_SCORE = SCORE_ARRAY[INDEX]

        // calculate running sum
        SUM = SUM + CURRENT_SCORE

        // check for maximum score
        IF CURRENT_SCORE > MAX
                MAX = CURRENT_SCORE
        ENDIF

        // check for minimum score
        IF CURRENT_SCORE < MIN
                MIN = CURRENT_SCORE
        ENDIF

END FOR

// calculate average score
AVERAGE = SUM / 5

Return to MAIN PROGRAM

// call letter grade subroutine
Call LETTER_GRADE_SUBROUTINE

// letter grade subroutine
LETTER_GRADE_SUBROUTINE

IF AVERAGE >= 90
        LETTER_GRADE = 'A'

ELSE IF AVERAGE >= 80
        LETTER_GRADE = 'B'
```

```
ELSE IF AVERAGE >= 70
        LETTER_GRADE = 'C'

ELSE IF AVERAGE >= 60
        LETTER_GRADE = 'D'

ELSE
        LETTER_GRADE = 'F'

ENDIF

Return to MAIN PROGRAM

// call output subroutine
Call OUTPUT_SUBROUTINE

// output subroutine
OUTPUT_SUBROUTINE

Display "Minimum Score = "
Display MIN

Display "Maximum Score = "
 Display MAX

Display "Average Score = "
Display AVERAGE

Display "Letter Grade = "
Display LETTER_GRADE

Return to MAIN PROGRAM

// stack operations
Before subroutines call:
        PUSH registers onto stack

Before returning:
        POP registers from stack

// save and restore register values
Save registers before modifying them
Restore registers before RET instruction

// stop program

HALT
```

A-2: Corrections:




During the initial compilation phase of the v1 codebase prototype, the assembler log flagged eleven cascading syntax failures originating from a critical misinterpretation of conditional branching mechanics. The initial draft attempted to execute grade range validation using a non-existent BRlt (Branch Less Than) macro-instruction. Because the LC-3 architecture natively processes condition codes using only negative, zero, and positive status flags, the compiler fails to parse BRlt as an opcode and instead interprets it as a program label declaration. This triggers a compounding domino effect across the assembly sequence: the actual targeted loop destinations (such as CHK_80, CHK_70, and CHK_60) are pushed into the opcode column, forcing the compiler to reject them as unrecognized instruction commands and flag subsequent lines as illegal attempts to redefine duplicate labels. Remedying this primary architectural limitation requires refactoring the code to utilize a native two's complement subtraction loop combined with a valid BRn opcode to evaluate negative flag states programmatically.

This compilation failure is further compounded by a position-dependent column alignment violation regarding the placement of label declarations within the source file. In the v1 prototype, several conditional destination markers were pushed into the indented instruction column rather than starting flush against the leftmost margin. Because the LC-3 assembler relies on rigid spatial rules, any character string beginning strictly in column 1 is registered as a global symbolic label, whereas text nested inside indented columns is evaluated as an executable opcode or directive. By failing to isolate these evaluation labels in column 1, the compiler misinterprets the structural markers as executable operations, resulting in immediate syntax errors. To completely eliminate these alignment bugs and achieve successful compilation, every logical target label was hard-aligned flush left against the leftmost margin, while all associated executable instructions and operands were moved forward using standardized tabbed indentation to restore correct hardware spatial parsing.

## B. **Development Process**

Architectural Source Formatting & Internal Documentation

The entire software implementation maintains structural integrity through a rigid conventions schema for internal labeling and inline code documentation. Every operational destination, conditional target block, loop boundary, and static data block is mapped to a unique, clear text

label that prevents overlapping symbols during the assembler's Pass 1 compilation sequence. Furthermore, every instruction cluster contains explicit inline comments documenting register assignments, tracking variables, and logical state changes. This documentation practice ensures that complex low-level manipulations remain readable, transparent, and scalable for multi-developer code audits.

## Modular Code Architecture & Inter-Subroutine Linkage

The application structure is built around a deeply modular framework consisting of four distinct subroutines to segregate logical processing tasks. Functional separation is achieved by utilizing explicit subroutine calls via JSR and JSRR instructions, allowing the core main program routine to delegate execution to isolated calculation environments. Control flow within these modules relies heavily on branching operations to handle both iterative loops and conditional checks. Iterative branching structures handle the multi-pass array processing sequences by decrementing a dedicated tracking register counter, while conditional branching monitors the zero and negative condition codes to determine score extremes and systematically evaluate the average grade boundaries.

## Low-Level Stack Frame Management & Context Protection

To prevent register conflicts and data corruption across subroutine boundaries, the application implements a strict runtime software stack. The software manages memory pointers dynamically, using a designated base register to step through the contiguous index boundaries of the allocated score array. Before a subroutine call diverts the program counter, explicit stack operations perform a context-protecting sequence by pushing active registers onto the data stack. Upon module completion, an exact inverse sequence executes a series of pop operations, restoring the original data values to their precise register configurations immediately before hitting the RET instruction. This strict save-restore policy guarantees that independent processing loops never clobber local variables.

Sections 2 and 3

## Part II- Functional Requirements

This program is designed to take 5 test scores from the user and calculate the minimum score, maximum score, average score, and letter grade equivalent.
The user will enter the test scores through the keyboard using console input. The program will store the scores into memory using an array. Since LC-3 reads characters as ASCII values, the program will convert the ASCII input into numerical integer values before performing calculations.

The program will loop through all 5 scores to

- Find the minimum score
- Find the maximum score
- Calculate the total sum
- Calculate the average score

After calculating the average, the program will determine the correct letter grade using conditional branching

Grade Scale

90-100 = A
80-89 = B
70-79= C
60-69 = D
0-59 = F

The program will display the following results on the console screen

Minimum score
Maximum score
Average score
Letter Grade

The application will use loops and branching instructions for program control and decision making. Pointer addressing will be used to move through the score array stored in memory.

The program will include subroutines such as

Input subroutine
Average calculation subroutine
Min/max calculation subroutine
Letter grade subroutine

The subroutines will use save and restore operations to protect register values during execution.

The program will also use stack operations with PUSH and POP instructions for temporary storage and subroutine management.
ASCII conversion operations will be implemented to convert keyboard input characters to integer values that can be used in arithmetic calculations.
The program will include proper labels, comments, memory allocation, arrays, constants, and LC-3 system calls for input and out

## Scope

### Phase 1- Program Setup

The first phase will include setting up the program origin, memory allocation, arrays, constants, labels, and stack initialization. Basic input and output systems calls will also be prepared.

### Phase 2- User input and Storage

The second phase will focus on accepting 5 test scores from the user through keyboard input. ASCII conversion operations will be implemented to convert character input into numerical value. The scores will then be stored in memory using pointer addressing.

### Phase 3- Score Processing

The third phase will implement the calculations for minimum score, maximum score, total score, and average score. Loops and conditional branching instructions will be used to process the array values.

### Phase 4- Letter Grade conversion

The fourth phase will determine the correct letter grade based on the calculated average score using conditional comparisons and branching instructions.

### Phase 5- Subroutines and Stack Operations

The fifth phase will implement subroutines for calculation and input/output operations. PUSH and POP stack operations will be added along with save-restore register functionality.

## Performance

The program is designed to process and display results efficiently using loops, branching, and subroutines.

The program will store only five test scores in memory, which minimizes memory usage and allows calculations to be completed quickly during execution.

All calculations for minimum score, maximum score, total score, average score, and letter grade will be completed immediately after the user enters the five scores. The application will use array storage and pointer addressing to reduce unnecessary memory operations.

Subroutines and stack operations will be used to organize the program while maintaining efficient register management through save and restore operations.

## Usability

The program will display prompts for the user when entering each of the five test scores. Input and output message will clearly identify what the user should enter and what results are being displayed.

The program is designed to accept numerical test score input only and process the values immediately after all scores are entered. Labels, comments, and subroutines will be organized clearly to improve readability.

## Documenting Requests for Enhancements

| date | Enhancement | Requested by | notes | priority | Release N/O |
|---|---|---|---|---|---|
| 5/22 | Update fixes on code | D’von | 11 issues | yes | 5/22 |
| | | | | | |
| | | | | | |

Code for the Working Model of our Program (Used in Part 1s Documentation):

```
;=================================================================
; LC-3 TEST SCORE CALCULATOR - MASTER BUILD
;=================================================================

        .ORIG   x3000

MAIN    ; Initialize stack pointer and variables
        LD      R6, STACK_INIT  ; Set R6 to x4000 (Stack Pointer)
        AND     R2, R2, #0      ; SUM = 0
        ST      R2, SUM

        ; Call input subroutine
        JSR     INPUT_SUB

        ; Initialize MIN and MAX using first score
        LEA     R5, SCORE_ARRAY ; Load base address of array
        LDR     R0, R5, #0      ; Load SCORE_ARRAY[0]
        ST      R0, MIN         ; MIN = SCORE_ARRAY[0]
        ST      R0, MAX         ; MAX = SCORE_ARRAY[0]

        ; Call calculation subroutine
        JSR     CALC_SUB

        ; Call letter grade subroutine
        JSR     GRADE_SUB

        ; Call output subroutine
        JSR     OUTPUT_SUB
```

```
    HALT                ; Stop program execution safely

;--- GLOBAL STORAGE / CONSTANTS ---
STACK_INIT  .FILL   x4000
SCORE_ARRAY .BLKW   5
SUM         .FILL   x0000
MIN         .FILL   x0000
MAX         .FILL   x0000
AVERAGE     .FILL   x0000
LETTER      .FILL   x0000
NEG_X30     .FILL   xFFD0       ; Two's complement of x30 (-48)
POS_X30     .FILL   x0030       ; ASCII offset for numbers (+48)

;=============================================================================
; SUBROUTINE: INPUT_SUB
;=============================================================================
;(This is the block that needed to be updated from the pseudocode/ original draft)
INPUT_SUB
    STR     R1, R6, #0      ; PUSH R1
    STR     R5, R6, #-1     ; PUSH R5
    STR     R7, R6, #-2     ; PUSH R7 (Safeguard return address from TRAPs)
    ADD     R6, R6, #-3     ; Move Stack Pointer down by 3

    AND     R1, R1, #0      ; INDEX = 0
    LEA     R5, SCORE_ARRAY ; Base array pointer

IN_LOOP ADD     R3, R1, #-5     ; Check if INDEX == 5
    BRzp    IN_DONE

    ; Prompt User
```

```
LEA     R0, PROMPT
TRAP    x22             ; PUTS

; Read multi-digit inputs (simplified sequence)
TRAP    x20             ; GETC (Read first digit)
TRAP    x21             ; OUT
LD      R4, NEG_X30
ADD     R0, R0, R4      ; Convert first digit to int

; Scale first digit by 10 (Iterative ADD)
ADD     R3, R0, #0      ; Copy value
AND     R4, R4, #0      ; Clear accumulator
ADD     R4, R4, R3      ; 1
ADD     R4, R4, R3      ; 2
ADD     R4, R4, R3      ; 3
ADD     R4, R4, R3      ; 4
ADD     R4, R4, R3      ; 5
ADD     R4, R4, R3      ; 6
ADD     R4, R4, R3      ; 7
ADD     R4, R4, R3      ; 8
ADD     R4, R4, R3      ; 9
ADD     R4, R4, R3      ; 10
ADD     R3, R4, #0      ; R3 = (Digit 1 * 10)

TRAP    x20             ; GETC (Read second digit)
TRAP    x21             ; OUT
LD      R4, NEG_X30
ADD     R0, R0, R4      ; Convert second digit to int
ADD     R0, R3, R0      ; TOTAL_SCORE = Tens + Units
```

```
    STR     R0, R5, #0      ; Store total into SCORE_ARRAY[INDEX]

    ; Newline formatting
    LEA     R0, NEWLINE
    TRAP    x22

    ADD     R5, R5, #1      ; Increment array address pointer
    ADD     R1, R1, #1      ; INDEX++
    BRnzp   IN_LOOP

IN_DONE ADD     R6, R6, #3      ; Move Stack Pointer back up
    LDR     R7, R6, #-2     ; POP R7
    LDR     R5, R6, #-1     ; POP R5
    LDR     R1, R6, #0      ; POP R1
    RET

PROMPT  .STRINGZ "\nEnter Test Score: "
NEWLINE .STRINGZ "\n"

;===============================================================================
; SUBROUTINE: CALC_SUB
;===============================================================================

CALC_SUB
    STR     R1, R6, #0      ; PUSH R1
    STR     R5, R6, #-1     ; PUSH R5
    ADD     R6, R6, #-2     ; Move Stack Pointer down by 2

    AND     R1, R1, #0      ; INDEX = 0
    LEA     R5, SCORE_ARRAY
```

```
        LD      R2, SUM         ; Load global sum tracking

CALC_LP ADD     R3, R1, #-5     ; Check if INDEX == 5
        BRzp    CALC_DN

        LDR     R0, R5, #0      ; CURRENT_SCORE = SCORE_ARRAY[INDEX]
        ADD     R2, R2, R0      ; SUM = SUM + CURRENT_SCORE

        ; Check for Maximum (CURRENT_SCORE > MAX)
        LD      R3, MAX
        NOT     R3, R3
        ADD     R3, R3, #1      ; Two's complement (-MAX)
        ADD     R3, R0, R3      ; CURRENT_SCORE - MAX
        BRnz    CHK_MIN         ; If not greater, skip to min
        ST      R0, MAX         ; MAX = CURRENT_SCORE

CHK_MIN ; Check for Minimum (CURRENT_SCORE < MIN)
        LD      R3, MIN
        NOT     R3, R3
        ADD     R3, R3, #1      ; Two's complement (-MIN)
        ADD     R3, R0, R3      ; CURRENT_SCORE - MIN
        BRzp    NEXT_EL         ; If not lower, skip
        ST      R0, MIN         ; MIN = CURRENT_SCORE

NEXT_EL ADD     R5, R5, #1      ; Advance array pointer
        ADD     R1, R1, #1      ; INDEX++
        BRnzp   CALC_LP

CALC_DN ST      R2, SUM         ; Save updated sum
```

```
        ; Calculate Average (SUM / 5) using subtraction loops
        AND     R3, R3, #0      ; Clear quotient register (AVERAGE)
AVG_LP  ADD     R2, R2, #-5     ; Subtract 5
        BRn     AVG_DN          ; If negative, division is complete
        ADD     R3, R3, #1      ; Increment average count
        BRnzp   AVG_LP
AVG_DN  ST      R3, AVERAGE     ; Store calculated average

        ADD     R6, R6, #2      ; Move Stack Pointer back up
        LDR     R5, R6, #-1     ; POP R5
        LDR     R1, R6, #0      ; POP R1
        RET

;===============================================================================
; SUBROUTINE: GRADE_SUB
;===============================================================================

GRADE_SUB
        STR     R0, R6, #0      ; PUSH R0
        STR     R1, R6, #-1     ; PUSH R1
        STR     R7, R6, #-2     ; PUSH R7
        ADD     R6, R6, #-3     ; Move Stack Pointer down by 3

        LD      R0, AVERAGE     ; Load score to analyze

        ; Check >= 90
        ADD     R1, R0, #-16
        ADD     R1, R1, #-16
        ADD     R1, R1, #-16
        ADD     R1, R1, #-16
```

```
        ADD     R1, R1, #-16
        ADD     R1, R1, #-10    ; Complete subtraction of 90
        BRn     CHK_80
        LD      R1, CHAR_A
        BRnzp   GR_STORE

CHK_80  ;Check >= 80
        ADD     R1, R0, #-16
        ADD     R1, R1, #-16
        ADD     R1, R1, #-16
        ADD     R1, R1, #-16
        ADD     R1, R1, #-16    ; Complete subtraction of 80
        BRn     CHK_70
        LD      R1, CHAR_B
        BRnzp   GR_STORE

CHK_70  ;Check >= 70
        ADD     R1, R0, #-16
        ADD     R1, R1, #-16
        ADD     R1, R1, #-16
        ADD     R1, R1, #-16
        ADD     R1, R1, #-6     ; Complete subtraction of 70
        BRn     CHK_60
        LD      R1, CHAR_C
        BRnzp   GR_STORE

CHK_60  ;Check >= 60
        ADD     R1, R0, #-16
        ADD     R1, R1, #-16
        ADD     R1, R1, #-16
```

```
        ADD     R1, R1, #-12    ; Complete subtraction of 60
        BRn     GET_F
        LD      R1, CHAR_D
        BRnzp   GR_STORE

GET_F   LD      R1, CHAR_F

GR_STORE ST     R1, LETTER      ; Save assigned letter character

        ADD     R6, R6, #3      ; Move Stack Pointer back up
        LDR     R7, R6, #-2     ; POP R7
        LDR     R1, R6, #-1     ; POP R1
        LDR     R0, R6, #0      ; POP R0
        RET

CHAR_A  .FILL   x0041           ; 'A'
CHAR_B  .FILL   x0042           ; 'B'
CHAR_C  .FILL   x0043           ; 'C'
CHAR_D  .FILL   x0044           ; 'D'
CHAR_F  .FILL   x0046           ; 'F'

;===============================================================================
; SUBROUTINE: OUTPUT_SUB
;===============================================================================

OUTPUT_SUB
        STR     R0, R6, #0      ; PUSH R0
        STR     R7, R6, #-1     ; PUSH R7 (Safeguard across nested helper JSR)
        ADD     R6, R6, #-2     ; Move Stack Pointer down by 2
```

```
; Print Minimum
LEA     R0, OUT_MIN
TRAP    x22
LD      R0, MIN
JSR     PRINT_NUM

; Print Maximum
LEA     R0, OUT_MAX
TRAP    x22
LD      R0, MAX
JSR     PRINT_NUM

; Print Average
LEA     R0, OUT_AVG
TRAP    x22
LD      R0, AVERAGE
JSR     PRINT_NUM

; Print Letter Grade
LEA     R0, OUT_LET
TRAP    x22
LD      R0, LETTER
TRAP    x21             ; Direct single char print
LEA     R0, NEWLINE
TRAP    x22

ADD     R6, R6, #2      ; Move Stack Pointer back up
LDR     R7, R6, #-1     ; POP R7
LDR     R0, R6, #0      ; POP R0
RET
```

```
OUT_MIN .STRINGZ "Minimum Score = "
OUT_MAX .STRINGZ "Maximum Score = "
OUT_AVG .STRINGZ "Average Score = "
OUT_LET .STRINGZ "Letter Grade  = "

;--- Helper Routine: Convert value to multi-character ASCII output ---

PRINT_NUM
      STR     R1, R6, #0
      STR     R2, R6, #-1
      STR     R7, R6, #-2     ; Nested call safeguard
      ADD     R6, R6, #-3

      AND     R1, R1, #0      ; Tens column counter
      ADD     R2, R0, #0      ; Working value copy

P_LOOP  ADD     R2, R2, #-10
      BRn     P_REM
      ADD     R1, R1, #1
      BRnzp   P_LOOP

P_REM   ADD     R2, R2, #10     ; Restore leftover remainder units
      LD      R4, POS_X30

      ADD     R0, R1, R4      ; Format tens digit to ASCII
      TRAP    x21
      ADD     R0, R2, R4      ; Format units digit to ASCII
      TRAP    x21
```

```
        LEA     R0, NEWLINE
        TRAP    x22

        ADD     R6, R6, #3
        LDR     R7, R6, #-2     ; Restore components
        LDR     R2, R6, #-1
        LDR     R1, R6, #0
        RET
        .END
```

(End of test code)